Lab 3: Information Retrieval

Session 1. Indexing documents with Woosh.

Goal

The lab session focuses on getting practical experience in index creation in Python, emphasizing the design of flexible document schemas and sophisticated indexing strategies. Students will explore Whoosh's field types (TEXT, KEYWORD, NUMERIC, DATETIME, ID, STORED) and learn to construct complex schemas that support diverse data representations, including full-text, structured, and metadata fields. The session covers indexing operations: creating indices with customizable schemas, implementing advanced preprocessing using Analyzers, Tokenizers, and Filters to normalize and enhance text representation, and performing dynamic document management through update and deletion operations. Students will develop skills in creating indexing pipelines to handle document collections, implement custom text analysis workflows, and build information retrieval systems that support complex querying and efficient document manipulation.

Bibliography

- https://pypi.org/project/Whoosh/
- https://whoosh.readthedocs.io/en/latest/index.html

INTRODUCTION

Whoosh is a library containing classes and functions for indexing text and searching on the created index. It allows the development of custom search engines.

Text indexing, the information stored for each term in each field, the parsing of search queries, the types of the allowed queries, the ranking algorithms, etc., are customisable, replaceable, and extensible.

- Whoosh uses only pure Python, so only the Python interpreter is needed.
- Whoosh uses, by default, Okapi BM25F (https://es.wikipedia.org/wiki/Okapi_BM25), as a ranking function, but it can be customized.
- Whoosh creates small indexes compared to many other libraries.
- All text indexed in Whoosh must be in Unicode.
- Whoosh allows you to associate and store Python objects with indexed documents

STEPS FOR INDEXING DOCUMENTS

1. The schema of the index must be created. The schema is the list of fields contained in the index. For each field must be specified the following information:

- The name of the field
- The type of the field: (ID, STORED, KEYWORD, TEXT, NUMERIC, BOOLEAN, DATETIME, NGRAM, NGRAMWORDS)
- whether or not to store the field with the index
- The analyzer to be used

2. The index is created on disk. For that, a directory to store it in must be specified. A name can be specified with the **indexname** parameter (by default 'MAIN')

```
ix = open_dir("index_dir")
```

3. A writer is created, and the documents are added to the index. It is not necessary to give a value to all the fields in each document.

```
writer.add_document(title="First document", path="/a",
content="This is the text of the first document")
```

4.Commit the changes.

```
writer.commit()
```

EXAMPLE 1

```
from whoosh.index import create_in
from whoosh.fields import ID, TEXT

schema = Schema(title=TEXT(stored=True), path=ID(stored=True),
content=TEXT)
ix = create_in("index_dir", schema)
writer = ix.writer()
writer.add_document(title="First document", path="/a",
content="This is the text of the first document")
writer.add_document(title="Second document", path="/b",
content="This is the text of the second document")
 .
 .
 .
writer.commit()
```

ACTIVITY

A dataset of news is provided in the file **MN-DS-news-classification.zip**, which contains 10,917 news articles with hierarchical news categories collected between 1 January 2019 and 31 December 2019. Once unzipped, you get a .csv file, in which each row represents a news article, and the fields represent the news properties (id, date, source, title, content..., etc.). The dataset was manually labeled based on a hierarchical taxonomy with 17 first-level and 109 second-level categories. More details about the dataset can be found in the published paper "*MN-DS: A Multilabeled News Dataset for News Articles Hierarchical Classification"*

Each news is presented as a row in the dataframe with the following fields:

Description of fields in the dataframe

- id: Unique identifier of the article.
- date: Date of the article release.
- source: Publisher information of the article.
- title: Title of the news article.
- content: Text of the news article.
- author: Author of the news article.
- url: Link to the original article.
- published_utc: Date of article publication in utc time.
- collection_utc: Date of article scraping in utc time.

- category_level_1: First level category of Media Topic NewsCodes's taxonomy.
- category_level_2: Second level category of Media Topic NewsCodes's taxonomy.

Write a program that indexes the provided dataset. It receives as its first argument the directory where the news are located and the directory where the index is stored as its second argument.

python LNR_indexer.py path_to_dataset index_directory

The index schema must incorporate the following mandatory fields: *id, title, content, category_level_1, source, date, published_utc*. However, it is important to note that the "*content*" field should not be stored within the index. Furthermore, specific field types must be enforced:

- ***id*** must be of ID type to facilitate updating and removing the news.
- ***category_level_1*** must be of KEYWORD type to facilitate efficient grouping and querying based on categories.
- ***published_utc*** and ***date*** must be of DATETIME type to accurately represent publication dates and times.

News articles should be indexed and grouped by their primary category (*category_level_1*). The program should display the number of articles within each category and the time it takes to

process all news. The program should also show the total time to index the entire dataset and the number of indexed news.

EXAMPLE 2

```
python LNR_indexer.py ./MN-DS-news-classification.csv ./ind
==============================
Category: "arts, culture, entertainment and media", total news 300
Category: "conflict, war and peace", total news 800
Category: "crime, law and justice", total news 500
Category: "disaster, accident and emergency incident", total news 500
Category: "economy, business and finance", total news 400
Category: "education", total news 607
Category: "environment", total news 600
Category: "health", total news 700
Category: "human interest", total news 600
Category: "labour", total news 703
Category: "lifestyle and leisure", total news 300
Category: "politics", total news 900
Category: "religion and belief", total news 800
Category: "science and technology", total news 800
Category: "society", total news 1100
Category: "sport", total news 907
Category: "weather", total news 400
==============================
Total of indexed news: 10917
Writing the index...
Total time: 76.1s. (1.3m.)
==============================.
```

Appendix

Converting timestamp and string to datetime objects

Use the appropriate function or method to convert timestamps into datetime objects. For example, in Python, you can use the *datetime.fromtimestamp()* method to convert a Unix timestamp to a datetime object.

```
from datetime import datetime
timestamp_date=1572554996
mydate =datetime.fromtimestamp(timestamp_date)
```

Employ datetime parsing methods to convert strings into datetime objects. Python's datetime.fromisoformat() function is handy for this purpose.

```
from datetime import datetime
strdate= "2019-10-31"
mystrdate = datetime.fromisoformat(strdate)
```

Customizing tokenization of documents

In Whoosh, an analyzer (also known as parser) is a function that receives a string and returns a list of tokens. Basically, an analyzer is a tokenizer and several filters. Whoosh splits by spaces the string (as the default tokenizer) and only applies lowercasing as the default filter. Whoosh also incorporates different tokenizers and filters, see the documentation:
https://whoosh.readthedocs.io/en/latest/analysis.html.

```
from whoosh.analysis import LowercaseFilter, RegexTokenizer
tokenizer = RegexTokenizer()
lfilter = LowercaseFilter()
tokens = tokenizer("These ARE the **things I want!")
for token in lfilter(tokens):
    print(token.text)
```

When creating a schema, you can specify which analyzer Whoosh should be used by Whoosh, for example:

```
my_analyzer = RegexTokenizer() | LowercaseFilter() |
StopFilter()

schema = Schema(content=TEXT(analyzer=my_analyzer))
```